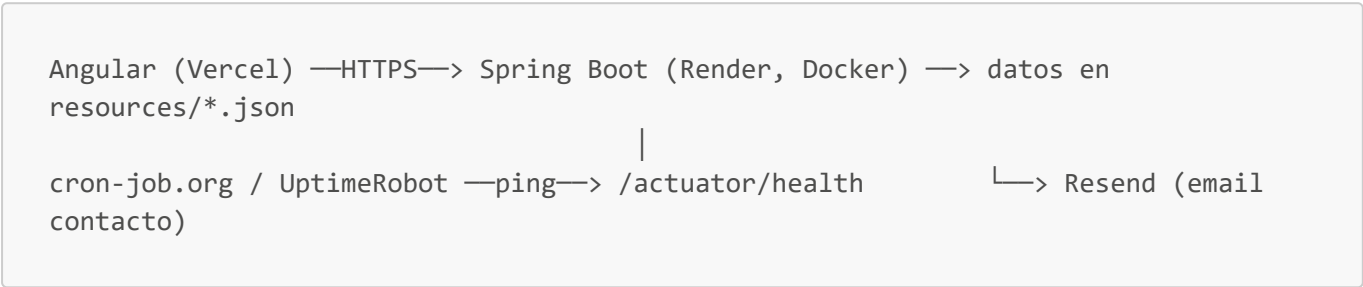


Guía: API del portfolio en Spring Boot (gratis, sin BBDD)

Para hacerla desde cero. Objetivo: `GET /api/proyectos` (datos embebidos en la API, sin base de datos) + `POST /api/contacto` (envía email) + keep-alive para que el free tier de Render no la duerma.

Arquitectura



1. Crear el proyecto — <https://start.spring.io>

- **Project:** Maven · **Language:** Java 21 · **Spring Boot:** 3.x (la última estable)
- **Packaging:** Jar

Dependencias (las librerías que me pediste)

Dependencia en Initializr	Artefacto Maven	Para qué
Spring Web	spring-boot-starter-web	Los endpoints REST y el <code>RestClient</code> (para llamar a Resend y para el autoping)
Validation	spring-boot-starter-validation	Validar el formulario de contacto (<code>@NotBlank</code> , <code>@Email...</code>)
Spring Boot Actuator	spring-boot-starter-actuator	Te regala <code>/actuator/health</code> → es el endpoint perfecto para el ping del keep-alive
Spring Boot DevTools (opcional)	spring-boot-devtools	Recarga en caliente en desarrollo
Lombok (opcional)	lombok	Menos boilerplate en los DTOs

NO necesitas: Spring Data JPA, ningún driver de BBDD, ni Spring Security (es una API pública de solo lectura + un POST con rate limit implícito del free tier).

Para el email NO hace falta librería si usas Resend: es una llamada HTTP con `RestClient` (incluido en Spring Web). Alternativa: `spring-boot-starter-mail` + una app password de Gmail, pero Resend es más simple y da 3.000 emails/mes gratis.

`@Scheduled` (para el autoping opcional) viene incluido en Spring, sin dependencia extra — solo requiere `@EnableScheduling` en la clase main.

2. Estructura de la API

```
src/main/java/...
├── controladores/
│   ├── ProyectosController.java    GET /api/proyectos?lang=es|en
│   │                               GET /api/experiencia?lang=...
│   │                               GET /api/formacion?lang=...
│   └── ContactoController.java     POST /api/contacto
├── servicios/
│   ├── DatosService.java           carga es.json / en.json de resources con
│   │                               Jackson
│   └── EmailService.java           llama a Resend con RestClient
├── config/
│   └── CorsConfig.java
└── KeepAlive.java                  (opcional, ver §6)

src/main/resources/
├── datos/es.json                  ← copiar la sección "datos" de
public/i18n/es.json               del proyecto Angular (misma estructura)
├── datos/en.json
└── application.properties
```

application.properties — DETALLE CLAVE PARA RENDER

```
# Render inyecta el puerto en la variable PORT. Sin esto, el deploy falla.
server.port=${PORT:8080}

resend.api-key=${RESEND_API_KEY:}
contacto.destino=amc.1994.mca@gmail.com
```

Leer los JSON de resources (Jackson, sin librería extra)

La idea: Jackson (incluido en Spring Web) lee `es.json/en.json` **una vez al arrancar** y los cachea en memoria; los endpoints solo sirven lo cacheado. No se lee el fichero en cada petición.

OJO: el fichero en `resources/datos/` contiene solo la sección `datos` del `i18n`, sin el envoltorio `"datos": {...}` ni las claves de UI (`nav`, `inicio`...). Es decir: `{ "proyectos": [...], "experiencia": [...], "formacion": [...] }`.

DTOs (records que reflejan la estructura del JSON):

```
public record Proyecto(
    String nombre, String tipo, String descripcion, String repositorio,
    String demo, List<String> tecnologias, List<String> caracteristicas,
```

```

        String fecha, String estado
    ) {}

    public record Experiencia(
        String puesto, String empresa, String fechaInicio, String fechaFin,
        List<String> tareas
    ) {}

    public record Formacion(
        String grado, String centro, String fechaInicio, String fechaFin,
        String competencias
    ) {}

    // Envuelve el fichero entero (es.json / en.json)
    public record Datos(
        List<Proyecto> proyectos,
        List<Experiencia> experiencia,
        List<Formacion> formacion
    ) {}

```

Servicio que carga los JSON al arrancar:

```

@Service
public class DatosService {

    private final ObjectMapper mapper;
    private final Map<String, Datos> porIdioma = new HashMap<>();

    public DatosService(ObjectMapper mapper) {
        this.mapper = mapper;
    }

    @PostConstruct
    void cargar() throws IOException {
        porIdioma.put("es", leer("datos/es.json"));
        porIdioma.put("en", leer("datos/en.json"));
    }

    private Datos leer(String ruta) throws IOException {
        // classpath: el fichero va dentro del jar (src/main/resources/)
        try (InputStream in = new ClassPathResource(ruta).getInputStream()) {
            return mapper.readValue(in, Datos.class);
        }
    }

    // Si piden un idioma que no existe, cae a español
    public Datos get(String lang) {
        return porIdioma.getOrDefault(lang, porIdioma.get("es"));
    }
}

```

Controlador:

```
@RestController
@RequestMapping("/api")
public class DatosController {

    private final DatosService datos;

    public DatosController(DatosService datos) {
        this.datos = datos;
    }

    @GetMapping("/proyectos")
    public List<Proyecto> proyectos(@RequestParam(defaultValue = "es") String
lang) {
        return datos.get(lang).proyectos();
    }

    @GetMapping("/experiencia")
    public List<Experiencia> experiencia(@RequestParam(defaultValue = "es") String
lang) {
        return datos.get(lang).experiencia();
    }

    @GetMapping("/formacion")
    public List<Formacion> formacion(@RequestParam(defaultValue = "es") String
lang) {
        return datos.get(lang).formacion();
    }
}
```

Detalles que evitan tropiezos:

- `ClassPathResource` lee desde dentro del jar (lo que ejecuta Render). `new File(...)` o rutas absolutas fallarían en el contenedor.
- Jackson mapea por nombre: las claves del JSON deben coincidir con los componentes del record. Una clave que falte (p.ej. `demo`, que solo tiene Portfolio) queda a `null` sin error. Records + Jackson funcionan de fábrica en Spring Boot 3, sin `@JsonProperty`.
- Cargar en `@PostConstruct` hace que un JSON malformado **falle al arrancar** (te enteras en el deploy, no cuando llega el primer GET).

CORS

Permitir solo el dominio propio y localhost:

```
@Configuration
public class CorsConfig implements WebMvcConfigurer {

    @Override
    public void addCorsMappings(CorsRegistry registry) {
        registry.addMapping("/api/**")
    }
}
```

```
        .allowedOrigins("https://www.codeadrianmc.dev",  
"http://localhost:4200")  
        .allowedMethods("GET", "POST");  
    }  
}
```

3. Dockerfile (multi-stage)

En la raíz del repo de la API:

```
# — Fase 1: compilar con Maven —————  
FROM maven:3.9-eclipse-temurin-21 AS build  
WORKDIR /app  
COPY pom.xml .  
RUN mvn dependency:go-offline -q          # cachea dependencias  
COPY src ./src  
RUN mvn package -DskipTests -q  
  
# — Fase 2: imagen final solo con el JRE —————  
FROM eclipse-temurin:21-jre-alpine  
WORKDIR /app  
COPY --from=build /app/target/*.jar app.jar  
EXPOSE 8080  
ENTRYPOINT ["java", "-jar", "app.jar"]
```

Y un `.dockerignore`:

```
target/  
.git/  
*.md
```

Probar en local antes de subir: `docker build -t portfolio-api . && docker run -p 8080:8080 portfolio-api`

4. Desplegar en Render

Qué necesitas: cuenta gratis en <https://render.com> (regístrate con GitHub, NO pide tarjeta) y el repo de la API subido a GitHub.

Pasos:

1. Dashboard → **New** → **Web Service**.
2. Conecta tu cuenta de GitHub y elige el repo `portfolio-api`.
3. Configuración:
 - **Language/Runtime:** Docker (lo detecta solo al ver el Dockerfile)
 - **Region:** Frankfurt (la más cercana)
 - **Instance Type:** Free

4. **Environment Variables** (sección Environment):

- `RESEND_API_KEY` = la API key de <https://resend.com> (cuenta gratis)
- 5. Create Web Service → primer build (~3-5 min) → te da <https://portfolio-api-XXXX.onrender.com>
- 6. Verificar: abre <https://...onrender.com/actuator/health> → debe responder `{"status":"UP"}`

Auto-deploy: cada push a `master` redespiega solo. Gratis.

Límites del free tier: 750 horas de instancia/mes (un servicio 24/7 = ~720h, entra justo), se duerme tras 15 min sin tráfico HTTP entrante, cold start ~30-60s al despertar.

5. Keep-alive con cron-job.org (o UptimeRobot)

El sleep de Render se evita con tráfico HTTP entrante periódico. **El ping externo es el mecanismo robusto** (puede despertar la API aunque ya se haya dormido).

Opción A — cron-job.org

1. Cuenta gratis en <https://cron-job.org>
2. **Create cronjob:**
 - URL: <https://tu-api.onrender.com/actuator/health>
 - Schedule: cada **10 minutos**
3. Listo. Gratis para siempre.

Opción B — UptimeRobot (extra: te avisa si la API se cae)

1. Cuenta gratis en <https://uptimerobot.com>
2. **Add New Monitor:** tipo HTTP(s), URL del health, intervalo **5 minutos**
3. Te llega email si la API deja de responder → monitor + keep-alive en uno.

6. Autoping opcional con @Scheduled (cinturón extra)

OJO: un `@Scheduled` que haga trabajo interno NO evita el sleep (Render solo cuenta tráfico HTTP entrante). El truco es pedirse a sí misma su URL pública:

```
@Component
public class KeepAlive {
    private final RestClient http = RestClient.create();

    @Scheduled(fixedRate = 5, timeUnit = TimeUnit.MINUTES)
    public void ping() {
        http.get()
            .uri("https://tu-api.onrender.com/actuator/health")
            .retrieve().toBodilessEntity();
    }
}
```

(+ `@EnableScheduling` en la clase main)

Limitación: si la API llega a dormirse (reinicio de Render, fallo de red en un ping), no puede despertarse sola. Por eso el ping externo del \$5 es el principal y esto, opcional.

7. Email con Resend — verificar el dominio (para no caer en spam)

El correo del formulario te lo mandas **a ti mismo** (el destinatario eres siempre tú: "Fulano quiere contactarte, su email es X, mensaje: Y"). Tú lo lees y le respondes desde tu Gmail. Por eso la entregabilidad es fácil... si verificas el dominio.

Pasos en Resend

1. Cuenta gratis en <https://resend.com> (3.000 emails/mes, sin tarjeta).
2. **Domains** → **Add Domain** → [codeadrianmc.dev](#).
3. Resend te muestra unos registros DNS (SPF, DKIM, y a veces DMARC). Añádelos en el panel donde gestionas el dominio (el mismo sitio donde lo apuntaste a Vercel):
 - un **TXT** para SPF
 - uno o varios **CNAME/TXT** para DKIM
4. Espera a que Resend marque el dominio como **Verified** (de minutos a un par de horas según el DNS).
5. **API Keys** → **Create** → esa key es la **RESEND_API_KEY** que pones en Render (\$4).

Detalles que importan

- **Sin dominio verificado:** Resend solo te deja enviar desde su dominio de pruebas ([onboarding@resend.dev](#)) y con límites. Sirve para probar, no para producción.
- **El from debe ser de tu dominio verificado**, p.ej. [contacto@codeadrianmc.dev](#). El **to** es tu Gmail. El email del visitante va en el **reply_to**, así al pulsar "Responder" en Gmail le contestas directamente a él.
- No necesitas crear un buzón [contacto@...](#) real: es solo la dirección remitente, no tiene que existir como cuenta.

Llamada a Resend (con el RestClient de Spring Web, sin librería extra)

```
http.post()
    .uri("https://api.resend.com/emails")
    .header("Authorization", "Bearer " + apiKey)
    .contentType(MediaType.APPLICATION_JSON)
    .body(Map.of(
        "from", "Portfolio <contacto@codeadrianmc.dev>",
        "to", List.of("amc.1994.mca@gmail.com"),
        "reply_to", visitante.email(),
        "subject", "Nuevo contacto de " + visitante.nombre(),
        "text", visitante.mensaje()
    ))
    .retrieve().toBodilessEntity();
```

8. Proteger el formulario contra bots

Si publicas **POST /api/contacto** sin protección, recibirás spam en tu propia bandeja. Defensa en capas, de más simple a más completa:

a) Honeypot (imprescindible, gratis, 0 fricción para el usuario)

Un campo oculto que un humano nunca rellena pero los bots sí. En Angular, un input escondido con CSS (no `type=hidden`, los bots lo detectan):

```
<input type="text" name="website" autocomplete="off" tabindex="-1"
      class="absolute -left-[9999px]" [(ngModel)]="honeypot" />
```

En la API, si ese campo viene relleno → descartar silenciosamente (responder 200 sin enviar nada, para no darle pistas al bot):

```
if (form.website() != null && !form.website().isBlank()) {
  return ResponseEntity.ok().build(); // es un bot, lo ignoramos
}
```

b) Rate limiting básico (recomendable)

Limita por IP, p.ej. máx. 3 envíos cada 10 min. Bucket4j es la librería estándar (`com.bucket4j:bucket4j-core`), o algo casero con un `Map<ip, timestamps>` para empezar.

c) Validación estricta (con la dependencia Validation del §1)

```
public record ContactoForm(
    @NotBlank @Size(max = 100) String nombre,
    @NotBlank @Email String email,
    @NotBlank @Size(max = 2000) String mensaje,
    String website // honeypot, debe venir vacío
) {}
```

Limitar longitudes evita que te llenen el email con megas de basura.

d) Captcha (opcional, solo si te llega spam pese a a/b/c)

Cloudflare Turnstile es gratis y menos molesto que reCAPTCHA. No lo pongas de entrada: el honeypot + rate limit frena al 99% de los bots sin estropear la experiencia.

9. Conectar Angular (cuando la API esté desplegada)

- `src/environments/` con `apiUrl` (`http://localhost:8080` dev / la URL de Render en prod).
- `provideHttpClient()` ya está configurado (lo usa Transloco).
- `ProyectosService`: pedir `GET {apiUrl}/api/proyectos?lang={idioma activo}` con **fallback a los JSON locales de i18n** si la API no responde (cubre el cold start).
- Skeleton/loading en la página de proyectos para el primer request.
- Formulario en contacto → `POST {apiUrl}/api/contacto`.
- En Vercel no hay que tocar nada.

Checklist

- ☐ Generar proyecto en start.spring.io (dependencias de la tabla del §1)
- ☐ `server.port=${PORT:8080}` en application.properties
- ☐ Copiar la sección `datos` de los JSON de i18n a resources/datos/
- ☐ Endpoints: proyectos, experiencia, formacion (con ?lang=) + contacto
- ☐ CORS (dominio propio + localhost:4200)
- ☐ Formulario: record con validación + honeypot + rate limit (§8)
- ☐ Cuenta en Resend + **verificar el dominio (DNS)** + probar envío de email (§7)
- ☐ `from` con dominio verificado, `to` tu Gmail, `reply_to` = email del visitante
- ☐ Dockerfile + probar `docker build` y `docker run` en local
- ☐ Subir repo a GitHub
- ☐ Render: New Web Service → Docker → Free → env var RESEND_API_KEY
- ☐ Verificar /actuator/health en la URL de Render
- ☐ cron-job.org o UptimeRobot apuntando al health cada 5-10 min
- ☐ Conectar Angular (environments + fallback local)